Experiment No. 5
Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment..
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment

Theory:

Monte Carlo Control

Monte Carlo methods are a class of algorithms used in reinforcement learning for estimating value functions and finding optimal policies. Unlike dynamic programming methods, Monte Carlo methods do not require a model of the environment and instead learn directly from experience by interacting with the environment.

Monte Carlo control combines Monte Carlo prediction with policy improvement to find the optimal policy directly from experience. After estimating the value function, the policy is improved by selecting actions that maximize the value function estimates.

Monte Carlo methods learn from complete episodes of experience. Here's the algorithm for Monte Carlo Control:

Input:

MDP: (S, A, P, R, γ), where

- S is the set of states.
- A is the set of actions.
- P is the transition probability matrix.
- R is the reward function.
- γ is the discount factor.

Number of episodes **num_episodes**.

Exploration rate **epsilon**.

Algorithm:

Optimal policy π.

Initialize:

Q-function: Q(s, a) for all s in S and a in A.

State-action visit count: N(s, a) for all s in S and a in A.

Policy π arbitrarily.

For each episode **from 1 to num_episodes**:

Generate an episode following π: (s1, a1, r1), (s2, a2, r2), ..., (sT, aT, rT).

Set G = 0.

For each time step **t** from T down to 1**:

Update return G: G = γ * G + rt+1.

If (st, at) has not been visited in this episode before:

Increment visit count: N(st, at) += 1.

Update action-value function: Q(st, at) = Q(st, at) + (1/N(st, at)) * (G - Q(st, at)).

Update policy: π(st) = argmax(Q(st, a)) for all a in A.

Return the optimal policy π.

Temporal Difference (TD) Learning

Temporal Difference learning methods update estimates based on the difference between current and future estimates. Here's the algorithm for TD Learning:

Input:

MDP: (S, A, P, R, γ), where

S is the set of states.

A is the set of actions.

P is the transition probability matrix.

R is the reward function.

γ is the discount factor.

Number of episodes **num_episodes**.

Step size **alpha**.

Exploration rate **epsilon**.

Algorithm:

Optimal policy π.

Initialize:

Q-function: Q(s, a) for all s in S and a in A.

Policy π arbitrarily.

For each episode **from 1 to num_episodes**:

Initialize state s as the starting state.

While s is not a terminal state:

Choose action a using policy derived from Q (e.g., ε-greedy).

Take action a, observe reward r and next state s'.

Update action-value function: $Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max(Q(s', a')) - Q(s, a))$.

Update policy: $\pi(s) = \text{argmax}(Q(s, a))$ for all a in A.

Set s = s'.

Return the optimal policy π.

These are high-level algorithms. The specific implementation details and additional considerations (like exploration strategy, eligibility traces, etc.) might vary based on the problem and requirements.

Implementation:

```
import numpy as np

import random


# Step 1: Grid World

grid_size = 4

num_states = grid_size * grid_size

actions = [0, 1, 2, 3]  # up, down, left, right


def next_state(s, a):

    row, col = divmod(s, grid_size)

    if a == 0 and row > 0: row -= 1

    elif a == 1 and row < grid_size - 1: row += 1

    elif a == 2 and col > 0: col -= 1

    elif a == 3 and col < grid_size - 1: col += 1

    return row * grid_size + col
```

```
def reward(s):
    return 10 if s == num_states - 1 else -1


gamma = 0.9
epsilon = 0.1
episodes = 500


# ------------------------
# MONTE CARLO CONTROL
# ------------------------
Q_mc = np.zeros((num_states, len(actions)))
returns = {}


for ep in range(episodes):
    episode = []
    state = 0


    # Generate episode
    while state != num_states - 1:
        if random.random() < epsilon:
```

```
            action = random.choice(actions)
        else:
            action = np.argmax(Q_mc[state])

        next_s = next_state(state, action)
        r = reward(next_s)
        episode.append((state, action, r))
        state = next_s

    G = 0
    visited = set()

    # Backward update
    for (s, a, r) in reversed(episode):
        G = gamma * G + r
        if (s, a) not in visited:
            visited.add((s, a))
            if (s, a) not in returns:
                returns[(s, a)] = []
            returns[(s, a)].append(G)
            Q_mc[s, a] = np.mean(returns[(s, a)])
```

```
# ------------------------
# TEMPORAL DIFFERENCE (Q-Learning)
# ------------------------
Q_td = np.zeros((num_states, len(actions)))
alpha = 0.1

for ep in range(episodes):
    state = 0

    while state != num_states - 1:
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q_td[state])

        next_s = next_state(state, action)
        r = reward(next_s)

        # TD Update
        Q_td[state, action] += alpha * (
```

```
        r + gamma * np.max(Q_td[next_s]) - Q_td[state, action]
    )

    state = next_s

# Step 3: Display Policies
policy_mc = np.argmax(Q_mc, axis=1)
policy_td = np.argmax(Q_td, axis=1)

print("Policy (Monte Carlo):")
print(policy_mc.reshape(grid_size, grid_size))

print("\nPolicy (TD Learning):")
print(policy_td.reshape(grid_size, grid_size))
```

Conclusion:

1. Difference between Monte Carlo and TD Learning:

Feature	Monte Carlo	TD Learning
Learning	After complete episode	Step-by-step
Update	Uses actual return	Uses estimated future reward
Speed	Slower	Faster
Requirement	Needs episode end	Works in ongoing tasks

---

2. Comparison (Advantages & Limitations):

Monte Carlo Control:

Advantages:

- Simple to understand
- Uses actual rewards (no approximation)

Limitations:

- Requires complete episodes
- Slow convergence

Temporal Difference (TD) Learning:

Advantages:

- Faster learning
- Works in real-time (online learning)
- Efficient for large problems

Limitations:

- Uses approximations (may introduce bias)
- Slightly complex